DSCI 571 Quiz 1

Frame task + data

Supervised learning: labeled (X,y) → fitted model → predict y for new X. Classification target = discrete class (even if coded 0/1); regression target = numerical quantity. One row/example; X shape `(n,d)`, `y (n,)`; one row/feature remains 2-D: `X.iloc[[0]]`, `df[["feature"]]`.
Before modeling: what is one example? target? features available **at prediction time**? intended use/error cost? Exclude target/post-outcome leakage and meaningless IDs; sensitive proxies require judgment. Predictive association/accuracy ≠ causation or appropriate deployment.

```
X = df.drop(columns="target")
y = df["target"]
X_train,X_test,y_train,y_test = train_test_split(
    X,y,test_size=.2,random_state=123,
    stratify=y)  # classification; omit for
continuous regression
```

Split test early; choice-driving EDA only on training. `random_state` reproduces, not validates. Respect time/groups/repeated subjects; random split can leak structure.

Estimator interface + metrics

API	Meaning
`Model(hyperparameters)`	unfitted estimator
`.fit(X_train,y_train)`	learns parameters; returns self
`.predict(X_new)`	predictions; does not fit
`.score(X_eval,y_eval)`	estimator-specific metric

Classification default here: accuracy = mean(ŷ=y); error = 1−accuracy. Imbalance/error costs can make accuracy misleading.
Regression tree/dummy default: $R^2 = 1 - \Sigma(y-\hat{y})^2 / \Sigma(y-\bar{y}_eval)^2$. 1 perfect; 0 equals constant prediction at **evaluation-set mean**; negative is legal/worse. R^2 is not percent correct; $1-R^2$ is not classification error. Constant/one-row y makes normal interpretation fail.

Baselines + decision trees

`DummyClassifier(strategy="most_frequent")`: learns training majority class, ignores X.
`DummyRegressor(strategy="mean"|"median")`: learns training constant. Compare all candidates using same data/folds/metric; baseline held-out score need not equal training score or 0 R^2.
Tree prediction: root → `feature <= threshold` left, > right → leaf class majority / regression mean. Fit greedily chooses feature/threshold to improve purity/error; not global exhaustive search. Numerical scale does not normally change tree partitions.

- parameter: split feature/threshold, leaf statistics—learned in `fit`.
- hyperparameter: `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_leaf_nodes`—chosen via validation.
- `max_depth` is an upper bound; actual tree may be shallower. Depth counts edges; stump depth=1.
- Greater capacity usually lowers train error, may first improve then harm validation. Identical X with conflicting y cannot be separated by a deterministic tree.
- Tree boundaries are axis-aligned partitions; training fit/feature importance is not causal evidence.

Generalization, Golden Rule, CV

Training learns parameters. Validation/CV selects features/preprocessing/model/hyperparameters. **Test is untouched until all choices are fixed**, then evaluated once. Looking at test plots/scores to revise choices turns it into development data.
k-fold CV: k fits per configuration; each fold validates once and trains k−1 times. `cross_validate` returns arrays: `train_score`, `test_score` (**validation fold**, not final test), `fit_time`, `score_time`; request train scores explicitly. CV clones estimators; refit selected workflow on all training data afterward.

```
cv = cross_validate(
    pipe, X_train, y_train, cv=5,
    return_train_score=True)
cv["train_score"].mean()
cv["test_score"].mean(), cv["test_score"].std()
pipe.fit(X_train,y_train)
final = pipe.score(X_test,y_test)  # once
```

Underfit: train and validation both poor vs baseline. Overfit: train high, validation much lower. Small mean differences relative to fold variability are weak evidence; similar results favor simpler justified workflow. CV fold scores overlap in training data; std is not automatically a CI/SE.

Similarity models: representation defines distance

Euclidean $d(u,v)=\sqrt{\Sigma_j (u_j-v_j)^2}$. Units, included features, encoding, and scale define "near." Large-scale/irrelevant dimensions can dominate; in high dimensions points become sparse and nearest/farthest distances less distinct (curse of dimensionality).
k-NN classifier: find k nearest training examples, vote; regressor: average targets. `weights="uniform"` equal influence; `"distance"` closer points matter more. Small k flexible/noisy/overfit; large k smooth/underfit. With uniform weights and k≈all training data → majority/global mean; with distance weights predictions remain query-dependent. Fit cheap/stores data; prediction/storage costly.
RBF SVM: similarity $K(x_i,x)=\exp(-\gamma||x_i-x||^2)$. Fit selects support vectors and learned contributions; not nearest-neighbor voting. - gamma small → broad influence/smooth; large → local/flexible. - C small → tolerates training mistakes/smoother; large → penalizes mistakes/flexible. - C and gamma interact: tune jointly inside training/CV. Scaling changes distances and gamma's meaning.

Transformers + leakage

Transformer `.fit(X)` learns representation parameters; `.transform(X)` applies them; `.fit_transform(X_train)` only for training. Validation/test/deployment use the already-fitted transformer. Anything learned—means, medians, categories, vocabulary, selected features—must be fold-local.
Standardization: `z = (x − μ_train) / σ_train`; transformed training features have mean ≈0 and SD ≈1. Validation/test reuse training μ/σ. It is not min–max scaling.

Feature	Treatment	Key trap
numeric	median impute → standardize	fit on training fold only
nominal	one-hot encode	no fake order; unseen → all learned indicators 0
ordinal	explicit ordered codes	codes also imply equal spacing
suitable binary	pass through	check semantics, not dtype only
ID/leakage/unwanted	drop	unlisted columns drop by default

Median imputation makes data usable; it does not recover missing truth or fix missingness bias. `drop="if_binary"` saves one indicator but all-zero may mean dropped category or unknown. High-cardinality one-hot can be sparse; memory efficiency does not create information.
Wrong: fit preprocessing on full data before split, or on all X_train before CV. Fold validation then affects learned transformation.

```
num = make_pipeline(
    SimpleImputer(strategy="median"),
    StandardScaler(),
)
onehot = OneHotEncoder(handle_unknown="ignore")
ordinal = OrdinalEncoder(categories=[level_order])
pre = make_column_transformer(
    (num, num_cols),
    (onehot, nom_cols),
    (ordinal, ord_cols),
    ("passthrough", bin_cols),
    ("drop", drop_cols),
)
pipe = make_pipeline(
    pre,
    KNeighborsClassifier(n_neighbors=5),
)
cv = cross_validate(
    pipe, X_train, y_train, cv=5,
    return_train_score=True,
)
```

Each fold fits a fresh imputer/scaler/encoder/model on its fold-training rows; validation only transforms/scores. Pass raw X + full pipeline to CV. Pipeline prevents specified-step leakage, but cannot stop a human repeatedly checking test.

Final decision checklist

1. Task/row/X/y/prediction time defined? 2. Test locked before EDA/choices? 3. Baseline and metric appropriate? 4. Train–validation pattern indicates capacity issue or data limitation? 5. Distance features correctly scaled/encoded; irrelevant dimensions removed for a reason? 6. Every learned preprocessing step inside pipeline/CV? 7. ColumnTransformer covers each feature exactly once and excludes target? 8. Selection uses validation, then refit, then one final test? 9. Claims limited to represented future data and actual error costs?